Technical Incident Investigation Assistant

Project - Business Applications with Agentic AI

Contents

Executive Summary: Conclusions

1. Investigation strategies adapt to the problem type.

- **Scenario 1:** Performs incident triage and dependency analysis.
- **Scenario 2:** Combines telemetry with research to explain unfamiliar errors.
- **Scenario 3:** Uses fleet-wide logs and Python for performance analysis.

2. Planning provides clear investigation direction. The agent consistently breaks problems into structured steps focused on relevant systems and evidence.

3. Tools work together in a transparent workflow. Log retrieval, dependency analysis, and quantitative tools support evidence-based results.

Executive Summary: Conclusions

4. Evidence drives conclusions.

Root-cause hypotheses are grounded in observed telemetry, dependency relationships, known error signatures, and service behavior.

5. Reports communicate findings clearly.

Each investigation produces a root-cause hypothesis, supporting evidence, blast-radius assessment, and recommended next actions.

Executive Summary: Business Recommendations

- Use the assistant as a **first-pass investigation accelerator** to reduce manual triage effort and surface likely causes faster.
- Maintain **known_error_patterns** repository to improve future diagnostic accuracy and consistency.
- Keep **service_dependencies** metadata current to strengthen dependency analysis and blast-radius assessment.
- Encourage **evidence-based investigations** by combining telemetry, quantitative analysis, and dependency context.
- Collect feedback on investigation quality and actionability to **continuously improve performance**.

Business Context

Company & Infrastructure

NovaTech Solutions is a mid-sized SaaS company providing cloud-based platforms for e-commerce and subscription businesses. Service reliability directly impacts revenue, so the IT operations team treats every incident seriously.

The company runs four production servers hosting microservices. These systems generate thousands of log entries per day across INFO, WARN, ERROR, and CRITICAL levels from services like payments and order management.

The Operational Bottleneck

This process typically takes 30–90 minutes per investigation. Most of that time goes into mechanical work running queries, switching between tools, reading documentation, formatting summaries rather than the actual reasoning the engineer brings to the problem.

Current Manual Investigation

When an alert fires or a customer escalation comes in, an on-call engineer has to investigate. The investigation process today involves:

1. Manually querying the log database to find relevant entries
2. Looking up unfamiliar error messages, stack traces, or vendor-specific error codes online to understand what they mean
3. Running ad-hoc analyses (filtering, aggregating, plotting) to spot patterns and outliers
4. Tracing requests across services to understand blast radius
5. Writing up findings in a structured report for the team

The AI-Powered Target State

The IT team wants an AI-powered assistant that handles the mechanical work and produces a starting-point investigation report in minutes, leaving the engineer to validate, refine, and act on the findings.

Objective

Build a single-agent system that acts as an **investigation assistant** for an on-call engineer. Given a natural-language incident description, the agent should:

- **Plan** the investigation steps before acting
- **Query the log database** to gather evidence (logs, statistics, service dependencies)
- **Search the web** when it encounters error messages, stack traces, or vendor codes it doesn't recognize from its existing knowledge
- **Run Python analyses** on retrieved data to compute statistics, identify outliers, or trace requests across services
- **Synthesize findings** into a clear investigation report with the evidence chain that produced them

The system is explicitly an **assistant**, not an autonomous detector. A human engineer reviews the output and decides on remediation. The goal is to compress the mechanical parts of the investigation, not to replace human judgment about what to do next.

Concretely, the agent should:

- Adapt its tool sequence to what the incident actually requires (different incidents need different evidence)
- Use all three tool categories - SQL, web search, Python, when the problem warrants
- Surface its reasoning at each step so the engineer can follow and validate
- Produce a final report that includes the root cause hypothesis, supporting evidence, and recommended next actions

The deliverable is the agent itself, demonstrated on three investigation scenarios that exercise different tool combinations.

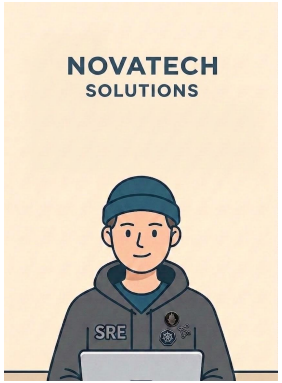
Solution Approach

Solution Approach: Empowering On-Call Engineers with Agentic AI

This solution is designed to support IT operations by integrating an AI Agent into the incident workflow, utilizing local telemetry and web access to build an evidence chain.

1. Knowledge Base Preparation

- Source of truth: A SQLite database (novatech_logs.db) containing raw server logs, anomaly thresholds, known error patterns, and service dependencies.
- This centralized structure allows the model to instantly contextualize system health and service topology.



Solution Approach

Solution Approach: Empowering On-Call Engineers with Agentic AI

2. Tool Overview

- **query_logs**: Retrieves filtered raw records.
- **get_log_stats**: Generates aggregated counts and distributions.
- **get_service_dependencies**: Retrieves upstream and downstream mappings.
- **get_known_patterns**: Retrieves documented error signatures and severities.
- **web_search**: Queries the internet for unfamiliar vendor codes or exceptions.
- **run_python_analysis**: Executes custom Pandas/Numpy scripts for statistical analysis.

3. LangGraph Workflow Implementation

- A state graph connects the planner, the tool-using agent, and a routing mechanism.
- **Planner Node**: Writes a structured investigation plan based on the incident trigger.
- **Agent Node**: Evaluates known data, calls the appropriate tool, observes results, and iterates.
- **Router Node**: Continues looping tool calls until the agent has sufficient evidence to write the final incident report.

Model Initialization

Model Used: "gpt-4o-mini"

A lighter variant of GPT-4 optimized for:

- **Low-latency inference** (faster responses in time-critical environments)
- **Cost-efficiency** for scaling in real-time applications
- **High-quality natural language generation**, suitable for question-answering tasks with contextual nuance.

Model Parameters & Reasoning

Parameter	Value	Reasoning
Temperature	0	Forces deterministic greedy decoding to guarantee highly repeatable execution traces across autonomous loops.

Data Loading

Database Used: “novatech_logs.db”

Preview Data:

```
=== LOG LEVEL DISTRIBUTION ===
  INFO       :  2993 entries
  WARN       :   168 entries
  ERROR      :    25 entries
  CRITICAL   :     9 entries

=== KNOWN ERROR PATTERNS ===
  OutOfMemoryError                Critical
  Connection refused              High
  Disk full                       Critical
  Segmentation fault              Critical
  Timeout exceeded                High

=== SERVICE DEPENDENCIES ===
  payment-api            -> auth-service           (synchronous)
  order-service          -> auth-service           (synchronous)
  order-service          -> payment-api            (synchronous)
  order-service          -> notification-service   (asynchronous)
  notification-service   -> email-relay            (synchronous)

=== SAMPLE ERROR / CRITICAL LOGS ===
  2025-06-15 02:14:33 | server_01 | CRITICAL | auth-service          | OutOfMemoryError: Java heap space exceeded limit...
  2025-06-15 02:15:01 | server_01 | ERROR    | auth-service          | Authentication failed for user admin@novatech.com after 5 at...
  2025-06-15 02:15:47 | server_01 | ERROR    | auth-service          | Authentication failed for user root@novatech.com after 5 att...
  2025-06-15 02:16:08 | server_01 | ERROR    | auth-service          | Connection pool exhausted: 50/50 connections in use for MySQ...
  2025-06-15 06:32:10 | server_02 | CRITICAL | payment-api           | Connection refused: Unable to reach payment-gateway on port ...
```

Defining Tools

Tool	What it helps the agent do
query_logs	Retrieve raw records from server_logs using filters such as server_id, service_name, log_level, timeframe, request_id, or message content
get_log_stats	Generate aggregated counts from server_logs grouped by server_id, service_name, and/or log_level
get_service_dependencies	Retrieve upstream_service, downstream_service, and dependency_type relationships from service_dependencies
get_known_patterns	Retrieve known error signatures, severity levels, and descriptions from known_error_patterns
web_search	Search for information about unfamiliar error messages, exception types, technologies, frameworks, or cloud service issues not covered by local data sources
run_python_analysis	Execute custom Pandas/Numpy analysis on data returned by query_logs using the provided DataFrame df and raw record list data

Agent Architecture

Planner Node

Defines the investigation sequence and logic based on the incident trigger.

Agent Node

Core reasoning engine that evaluates evidence and selects appropriate tool operations.

Tool Execution Layer

Executes SQL queries, web searches, and Python-based quantitative data analysis.

Routing Logic

Manages loop execution, determining whether to continue gathering evidence or finalize.

Agent State

Central repository holding conversation history, investigation triggers, and the current plan.

LangGraph Investigation Workflow

Planner Prompt:

""" You are a Site Reliability Engineer (SRE) planning a log investigation.

When given an alert or customer issue, your job is to write a step-by-step plan on how to investigate it using the tools below.

AVAILABLE TOOLS:

1. query_logs: Retrieve raw records from server_logs using filters such as server_id, service_name, log_level, timeframe, request_id, or message content.

2. get_log_stats: Generate aggregated counts from server_logs grouped by server_id, service_name, and/or log_level.

3. get_service_dependencies: Retrieve upstream_service, downstream_service, and dependency_type relationships from service_dependencies.

4. get_known_patterns: Retrieve known error signatures, severity levels, and descriptions from known_error_patterns.

5. web_search: Search for information about unfamiliar error messages, exception types, technologies, frameworks, or cloud service issues not covered by local data sources.

6. run_python_analysis: Execute custom Pandas/Numpy analysis on data retrieved from query_logs using the provided DataFrame `df` and raw record list `data`. SYSTEM THRESHOLDS: {threshold_lines}

LangGraph Investigation Workflow

Planner Prompt:

RULES FOR PLANNING:

- Pass data between tools: The `query_logs` tool returns a flat list of records. Pass this list directly into the `data` argument of `run_python_analysis`.

- Be efficient: If comparing multiple servers or services, pull all the relevant data in a single `query_logs` call rather than doing it one by one.

- Python outputs: Python scripts must use `print()` to return results (e.g., `print(df.groupby('server_id').mean().to_string())`).

CRITICAL TIME FORMATTING RULE:

- NEVER use a "T" or "Z" in timestamp filters (e.g., do NOT use '2025-06-15T06:30:00Z').

- ALWAYS use the standard format with a space: 'YYYY-MM-DD HH:MM:SS' (e.g., '2025-06-15 06:30:00').

YOUR OUTPUT:

Write a numbered plan with 4 to 7 steps.

For each step, specify exactly WHICH tool you will use and WHAT you are looking for.

Be specific about timeframes, services, or errors mentioned in the trigger."""

LangGraph Investigation Workflow

Agent System Prompt:

""" You are a Site Reliability Engineering (SRE) investigation assistant.

An on-call engineer has asked you to investigate a system incident.

You investigate by calling tools one at a time. Think step-by-step: look at what you know, decide what evidence you need next, call the right tool, observe the results, and then decide on your next step.

Use these standard operational thresholds to evaluate system health: {threshold_lines}

HOW TO USE YOUR TOOLS:

1. SQL Tools (query_logs, get_log_stats, get_service_dependencies, get_known_patterns):

- Use these to gather direct evidence from the database.

- Note: `query_logs` returns a simple, flat JSON list of log records.

- When passing `log_level`, use a simple string (e.g., "WARN,ERROR,CRITICAL").

2. Web Search (web_search):

- Use this if you encounter a specific vendor error code, cloud exception, or framework stack trace that is NOT in the `get_known_patterns` list. Do not guess look it up!

LangGraph Investigation Workflow

Agent System Prompt:

3. Python Analysis (run_python_analysis):

- Use this to compute statistics, compare servers, or find averages.

- You must explicitly pass the flat list of records you got from `query_logs` into the `data` parameter.

- A pandas DataFrame named `df` is automatically created for you to use in your code.

- CRITICAL: Your Python script MUST use `print()` to output the results (e.g., `print(df.groupby('server_id').mean().to_string())`). If you do not use `print()`, the output will be blank.

WHEN YOU ARE DONE INVESTIGATING:

Stop calling tools and generate a final investigation report with exactly these 5 sections:

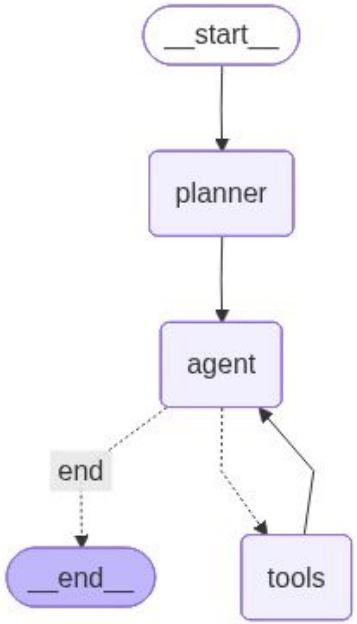
1. Summary of findings (2-3 sentences), 2. Root cause hypothesis (State your confidence: High, Medium, or Low), 3. Evidence chain (Which tool calls led to your conclusion), 4. Affected services / Blast radius, 5. Recommended next actions for the on-call engineer

If the evidence is ambiguous or incomplete, be honest about your uncertainty."""

LangGraph Investigation Workflow

Routing Logic:
If the agent requested a tool, run the tool; otherwise, stop the workflow and return the report.

Visualize Workflow:



Investigation Execution : Scenario 1

Investigation Trigger Prompt:

"On-call alert at 06:35 on 2025-06-15: payment portal response times have been elevated since around 06:30. Multiple users have reported checkout failures. Investigate what is going on, identify the most likely root cause, and recommend next steps."

Agent Investigation Plan:

```
======================================================================
INITIATING INVESTIGATION
======================================================================
Trigger: On-call alert at 06:35 on 2025-06-15: payment portal response times have been elevated since around 06:30. Multiple users have reported checkout failures. Investigate what is going on, identify the most likely root
======================================================================

----------------------------------------------------------------------
STEP 1: NODE = PLANNER
----------------------------------------------------------------------

PLAN:
1. **query_logs**: Retrieve raw records from `server_logs` for the payment portal service. Use filters for `service_name` as "payment_portal", `timeframe` from '2025-06-15 06:30:00' to '2025-06-15 06:40:00', and `log_level`

2. **run_python_analysis**: Analyze the retrieved log data to calculate the average and maximum `response_time_ms` for the payment portal service. Check for any instances where `response_time_ms` exceeds the critical thresh

3. **get_log_stats**: Generate aggregated counts from `server_logs` grouped by `log_level` for the payment portal service within the same timeframe ('2025-06-15 06:30:00' to '2025-06-15 06:40:00'). This will help determine

4. **get_known_patterns**: Retrieve known error signatures and descriptions from `known_error_patterns` to see if any of the errors or warnings from the logs match known issues. This can provide insights into potential root

5. **get_service_dependencies**: Retrieve the service dependencies for the payment portal service to identify any upstream or downstream services that might be affecting its performance. This will help determine if the issu

6. **web_search**: If any unfamiliar error messages or exception types are found in the logs that do not match known patterns, perform a web search to gather more information about these errors. This can provide additional

7. **Recommendation**: Based on the findings from the above steps, recommend next steps. This could include actions such as restarting services, scaling resources, or addressing specific errors identified in the logs.
```

Investigation Execution : Scenario 1

Final Investigation Report:

```
======================================================================
FINAL INVESTIGATION REPORT
======================================================================
### Summary of Findings
The payment portal has been experiencing elevated response times and checkout failures since 06:30 on 2025-06-15. Logs indicate a critical issue with the `payment-api` service, specifically a "Connection refused" error when

### Root Cause Hypothesis
**High Confidence**: The root cause is likely a connectivity issue between the `payment-api` and the `payment-gateway`, as indicated by the "Connection refused" error. This has caused a backlog of requests and high CPU and m

### Evidence Chain
1. **Query Logs**: Retrieved logs from `payment-api` showing elevated response times, retry storms, and a critical "Connection refused" error.
2. **Service Dependencies**: Confirmed `payment-api` depends on `payment-gateway` for processing payments.
3. **Known Patterns**: Identified "Connection refused" as a known high-severity issue related to connectivity problems.

### Affected Services / Blast Radius
- **Primary**: `payment-api` is directly affected, causing checkout failures.
- **Secondary**: Potential impact on `order-service` due to its dependency on `payment-api`.

### Recommended Next Actions
1. **Immediate**: Investigate and resolve the connectivity issue between `payment-api` and `payment-gateway`. Check network configurations, firewall settings, and service availability.
2. **Monitoring**: Increase monitoring on `payment-api` and `payment-gateway` to detect similar issues early.
3. **Resource Management**: Consider scaling resources for `payment-api` to handle retry storms more gracefully in the future.

----------------------------------------------------------------------
EVIDENCE CHAIN (Tool Calls)
----------------------------------------------------------------------

1. query_logs(service_name=payment-api, start_time=2025-06-15 06:30:00, end_time=2025-06-15 06:35:00, log_level=WARN,ERROR,CRITICAL)
    -> [   {     "log_id": 2700,     "timestamp": "2025-06-15 06:30:13",     "server_id": "server_02",     "log_level": "WARN",     "service_name": "payment-api",     "message": "User-facing latency spike: c...

2. get_service_dependencies(service_name=payment-api)
    -> [   {     "upstream_service": "payment-api",     "downstream_service": "auth-service",     "dependency_type": "synchronous"   },   {     "upstream_service": "order-service",     "downstream_service": ...

3. get_known_patterns()
    -> [   {     "pattern_keyword": "OutOfMemoryError",     "severity": "Critical",     "description": "JVM or process has exhausted available heap memory"   },   {     "pattern_keyword": "Disk full",     "s...

======================================================================
Total tool calls: 3
======================================================================
```

Investigation Execution : Scenario 1

Observations:

1. Successfully narrowed the investigation to the reported incident window and retrieved warning, error, and critical telemetry from payment-api, allowing rapid identification of the failure sequence.
2. Found direct evidence linking the customer-facing symptoms to backend infrastructure issues, including checkout timeout events and a critical "Connection refused" error involving the downstream payment-gateway.
3. Used service dependency data to validate that payment-gateway is a critical dependency of payment-api, strengthening the root-cause hypothesis and ruling out a purely application-level failure.
4. Cross-referenced observed errors against known operational failure patterns to increase confidence in the diagnosis without requiring broader exploratory searches.
5. Produced a focused incident report that connected user impact, probable root cause, affected services, and remediation steps, reducing the amount of manual triage required from the on-call engineer.

Investigation Execution : Scenario 2

Investigation Trigger Prompt:

"The reporting-service on server_03 logged a CRITICAL error around 08:45 on 2025-06-15 that the team hasn't seen before - something about 'SlowDown 503'. Pull the relevant log entries, work out what the error actually means, whether it indicates an issue on our side or on the upstream provider's side, and recommend mitigation steps."

Agent Investigation Plan:

```
======================================================================
INITIATING INVESTIGATION
======================================================================
Trigger: The reporting-service on server_03 logged a CRITICAL error around 08:45 on 2025-06-15 that the team hasn't seen before - something about 'SlowDown 503'. Pull the relevant log entries, work out what the error actua
======================================================================

----------------------------------------------------------------------
STEP 1: NODE = PLANNER
----------------------------------------------------------------------

PLAN:
1. **Query Logs**: Use the `query_logs` tool to retrieve all log entries for the reporting-service on server_03 with a log level of CRITICAL around the time of the incident. Set the timeframe from '2025-06-15 08:40:00' to '

2. **Get Known Patterns**: Use the `get_known_patterns` tool to check if the 'SlowDown 503' error matches any known error signatures. This will help determine if this is a recognized issue with documented severity and descr

3. **Web Search**: If the 'SlowDown 503' error is not found in known patterns, use the `web_search` tool to find information about this error message. Look for its meaning, common causes, and whether it is typically an issu

4. **Get Service Dependencies**: Use the `get_service_dependencies` tool to identify any upstream services that the reporting-service depends on. This will help determine if the issue might be related to an upstream provid

5. **Run Python Analysis**: Use the `run_python_analysis` tool to analyze the retrieved log data for patterns or anomalies, such as consecutive occurrences of the error or related metrics like response time, CPU, or memory

6. **Recommend Mitigation Steps**: Based on the findings from the previous steps, draft a recommendation for mitigation. If the issue is on our side, suggest changes to the reporting-service or server configuration. If it's
```

Investigation Execution : Scenario 2

Final Investigation Report:

```
======================================================================
FINAL INVESTIGATION REPORT
======================================================================
### Summary of Findings
The `reporting-service` on `server_03` encountered a CRITICAL error at 08:45 on 2025-06-15, related to an S3 client error: "SlowDown (503) - Please reduce your request rate." This error indicates that the request rate to Ama

### Root Cause Hypothesis
**Confidence: High**
The root cause of the error is the high request rate to Amazon S3, which triggered the "503 Slow Down" error. This is an issue on our side, as it relates to how our application is interacting with S3, rather than a fault wit

### Evidence Chain
1. **Log Retrieval**: Queried logs from `server_03` for the `reporting-service` around the time of the incident, revealing the "SlowDown (503)" error.
2. **Known Patterns Check**: Verified that this error was not in the known error patterns.
3. **Web Search**: Conducted a web search to understand the "SlowDown 503" error, confirming it is due to high request rates to S3.

### Affected Services / Blast Radius
The issue affects the `reporting-service` on `server_03`, specifically its interactions with Amazon S3. Other services may be indirectly affected if they rely on data from the `reporting-service`.

### Recommended Next Actions for the On-Call Engineer
1. **Implement Exponential Backoff**: Modify the application to implement exponential backoff for retries when encountering S3 throttling errors.
2. **Monitor Request Rates**: Set up monitoring to track request rates to S3 and identify patterns that lead to throttling.
3. **Optimize Request Patterns**: Review and optimize the request patterns to S3, possibly by distributing requests more evenly over time or reducing the overall request rate.
4. **Load Testing**: Conduct load testing to ensure the system can handle peak loads without exceeding S3's request limits.

----------------------------------------------------------------------
EVIDENCE CHAIN (Tool Calls)
----------------------------------------------------------------------

1. query_logs(server_id=server_03, service_name=reporting-service, log_level=CRITICAL, start_time=2025-06-15 08:40:00, end_time=2025-06-15 08:50:00)
    -> [   {     "log_id": 2767,     "timestamp": "2025-06-15 08:45:22",     "server_id": "server_03",     "log_level": "CRITICAL",     "service_name": "reporting-service",     "message": "S3 client error: S...

2. get_known_patterns()
    -> [   {     "pattern_keyword": "OutOfMemoryError",     "severity": "Critical",     "description": "JVM or process has exhausted available heap memory"   },   {     "pattern_keyword": "Disk full",     "s...

3. web_search(query=S3 SlowDown 503 error, max_results=5)
    -> [   {     "title": "Troubleshoot HTTP 5xx errors from Amazon S3 | AWS re:Post",     "url": "https://repost.aws/knowledge-center/http-5xx-errors-s3",     "snippet": "A high request rate or a sudden inc...

======================================================================
Total tool calls: 3
======================================================================
```

Investigation Execution : Scenario 2

Observations:

1. Successfully retrieved the critical incident log from reporting-service and isolated the unfamiliar "SlowDown (503)" error that triggered the investigation.
2. Verified that the error signature was not present in the internal known-pattern database, creating a legitimate need for external investigation rather than relying solely on local knowledge.
3. Correctly used web_search to determine that "SlowDown (503)" is an Amazon S3 throttling response generated when request rates exceed service limits.
4. Distinguished between an application defect and an upstream dependency constraint, concluding that the immediate failure originated from interactions with S3 rather than an internal reporting-service bug.
5. Combined evidence from local telemetry and external documentation to build a high-confidence root-cause hypothesis and explain the operational meaning of an otherwise unfamiliar error message.
6. Produced actionable mitigation guidance centered on request-rate management, retry behavior, and backoff strategies, giving engineers clear next steps for reducing the likelihood of future throttling events.

Investigation Execution : Scenario 3

Investigation Trigger Prompt:

"Something felt off across the platform last week (2025-06-09 to 2025-06-15). Latency complaints were coming from multiple teams, but nobody could identify a single outage or failing service. Pull the relevant telemetry, compare behavior across servers and services, identify what stood out from the norm acting as a data scientist and explain what the evidence suggests were the biggest contributors to the slowdown."

Agent Investigation Plan:

```
======================================================================
INITIATING INVESTIGATION
======================================================================
Trigger: Something felt off across the platform last week (2025-06-09 to 2025-06-15). Latency complaints were coming from multiple teams, but nobody could identify a single outage or failing service. Pull the relevant teler
======================================================================

----------------------------------------------------------------------
STEP 1: NODE = PLANNER
----------------------------------------------------------------------

PLAN:
1. **Query Logs for Latency Data**: Use the `query_logs` tool to retrieve records from `server_logs` for the timeframe '2025-06-09 00:00:00' to '2025-06-15 23:59:59'. Filter by `log_level` to include only logs with latency-

2. **Run Python Analysis for Latency Patterns**: Use the `run_python_analysis` tool with the data retrieved in step 1. Analyze the `response_time_ms` to identify any patterns or anomalies. Specifically, look for average res

3. **Get Log Stats for System Resource Usage**: Use the `get_log_stats` tool to generate aggregated counts of logs grouped by `server_id` and `log_level` for `cpu_usage_percent` and `memory_usage_percent`. This will help ic

4. **Run Python Analysis for Resource Usage Patterns**: Use the `run_python_analysis` tool with the data from step 3. Analyze `cpu_usage_percent` and `memory_usage_percent` to identify any patterns or anomalies. Look for ir

5. **Get Service Dependencies**: Use the `get_service_dependencies` tool to retrieve upstream and downstream service relationships. This will help understand if any service dependencies might have contributed to the latency

6. **Get Known Error Patterns**: Use the `get_known_patterns` tool to retrieve known error signatures and descriptions. Compare these with any error messages or patterns identified in the logs to see if any known issues cou

7. **Summarize Findings**: Based on the analysis from steps 2, 4, and 5, summarize the evidence to explain what the biggest contributors to the slowdown were. Consider both latency and resource usage patterns, as well as ar
```

Investigation Execution : Scenario 3

Final Investigation Report:

```
======================================================================
FINAL INVESTIGATION REPORT
======================================================================
### Summary of Findings
During the week of June 9 to June 15, 2025, the platform experienced significant latency issues, particularly affecting the `payment-api`, `auth-service`, and `order-service`. These services showed elevated response times an

### Root Cause Hypothesis
**Confidence Level: High**
The root cause of the slowdown appears to be resource exhaustion and performance degradation in the `payment-api` and `auth-service`, particularly on `server_01` and `server_02`. These services experienced high response time

### Evidence Chain
1. **Log Statistics**: Aggregated log counts showed high activity in `payment-api` and `order-service`.
2. **Log Analysis**: Detailed logs revealed frequent warnings and errors related to high response times and resource usage.
3. **Python Analysis**: Statistical analysis confirmed that `payment-api` and `auth-service` had the highest average response times and resource usage, with `server_01` and `server_02` being the most affected.

### Affected Services / Blast Radius
- **Primary Services**: `payment-api`, `auth-service`, `order-service`
- **Secondary Impact**: `notification-service`, `user-service`, `reporting-service`
- **Servers**: `server_01`, `server_02`, `server_03`, `server_04`

### Recommended Next Actions
1. **Resource Optimization**: Investigate and optimize resource allocation for `payment-api` and `auth-service` to prevent CPU and memory exhaustion.
2. **Load Balancing**: Consider redistributing load across servers to alleviate pressure on `server_01` and `server_02`.
3. **Performance Monitoring**: Implement enhanced monitoring for early detection of resource bottlenecks and latency spikes.
4. **Incident Review**: Conduct a post-mortem to identify specific triggers and prevent recurrence.

----------------------------------------------------------------------
EVIDENCE CHAIN (Tool Calls)
----------------------------------------------------------------------

1. get_log_stats(group_by=service_name, start_time=2025-06-09 00:00:00, end_time=2025-06-15 23:59:59)
    -> [   {     "service_name": "payment-api",     "count": 577   },   {     "service_name": "order-service",     "count": 541   },   {     "service_name": "auth-service",     "count": 532   },   {     "ser...

2. query_logs(start_time=2025-06-09 00:00:00, end_time=2025-06-15 23:59:59, log_level=WARN,ERROR,CRITICAL)
    -> [   {     "log_id": 145,     "timestamp": "2025-06-09 10:15:01",     "server_id": "server_03",     "log_level": "WARN",     "service_name": "reporting-service",     "message": "Server-level performanc...

3. run_python_analysis(data=[{'log_id': 145, 'timestamp': '2025-06-09 10:15:01', code=import pandas as pd

# Calculate average response )
    -> {   "output": "Service Stats:\n                      response_time_ms  cpu_usage_percent  memory_usage_percent\nservice_name                                                                                \nauth-s...

======================================================================
Total tool calls: 3
```

Investigation Execution : Scenario 3

Observations:

1. Identified payment-api, auth-service, and order-service as the primary contributors to platform degradation, with server_01 and server_02 showing the strongest indicators of sustained performance pressure.
2. Determined that the impact extended beyond a single service, affecting multiple application components and confirming a broad operational issue rather than an isolated incident.
3. Produced an evidence-based root-cause hypothesis linking elevated latency to sustained resource pressure and performance degradation observed across the most affected services and servers.
4. Generated a clear blast-radius assessment and prioritized remediation recommendations, reducing the amount of manual investigation required from infrastructure engineers.
5. Demonstrated successful data flow between tools by passing fleet-wide telemetry from query_logs directly into run_python_analysis, enabling automated aggregation and ranking of affected services and infrastructure components.

Great Learning

Power Ahead!